Chapter 2: MySQL Database Installation and Setup

You will use MySQL as the database for this project. This chapter shows you how to set up your local development server, how to access it in Flask, and basic operations.

Installing MySQL

You will only need to go through this installation once. After that, you will create a new database for each project.

Windows Installation

1. Go to dev.mysql.com/downloads.
2. Download 'MySQL Installer for Windows' (`mysql-installer-community`).
3. Run the installer and select 'Custom' installation.
4. Select 'MySQL Server' ***and*** 'MySQL Workbench' from the products list.
5. Follow the wizard to complete installation.
6. Set a root password and remember it!

> ⚠ **If you did not enable the option to automatically start the server with every device boot, you will need to manually start it when you need it**
>
> - Open the Windows "Run" dialog by pressing the Windows key + R
> - Type `services.msc` and press "ok"
> - Find `MySQL80` in that list, right-click on it, and click "start"

Mac Installation

1. Go to dev.mysql.com/downloads.
2. Select macOS and download the DMG archives for `MySQL Community Server` ***and*** `MySQL Workbench` .
3. Double-click the downloaded DMG file.
4. Run the installer package (`.pkg`).
5. Follow the installation wizard.
6. **IMPORTANT:** Save the temporary root password shown at the end!

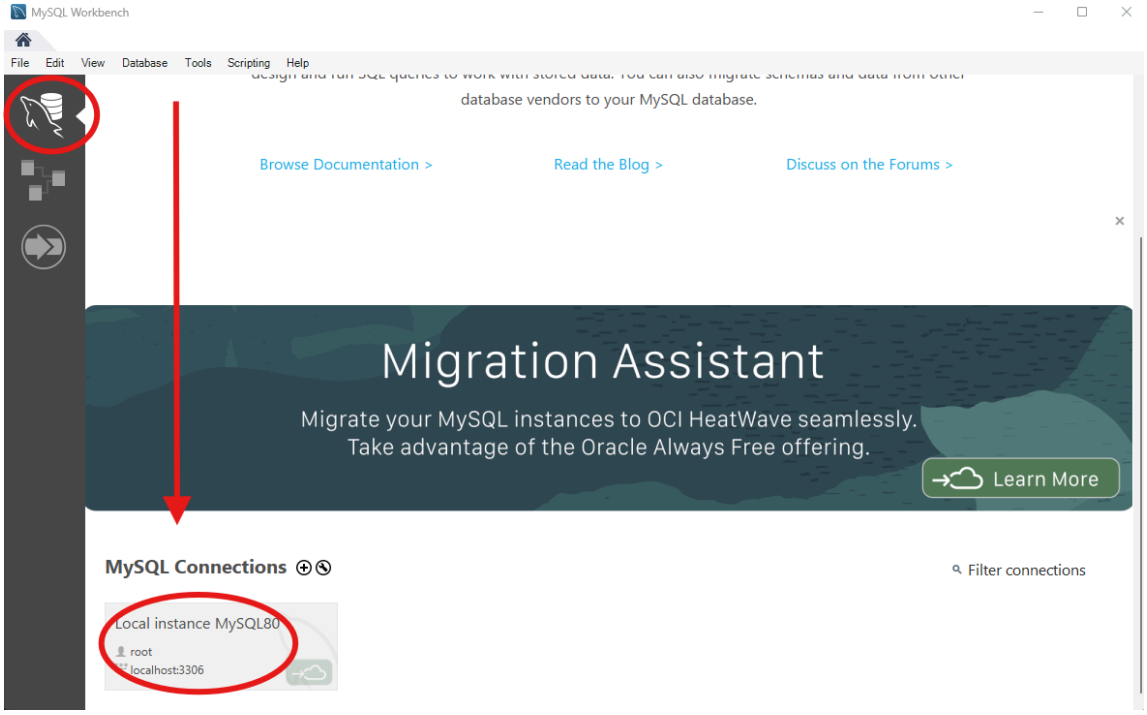
If you lose your MySQL root password, you have to reset it through a recovery process.

What are MySQL Server and Workbench?

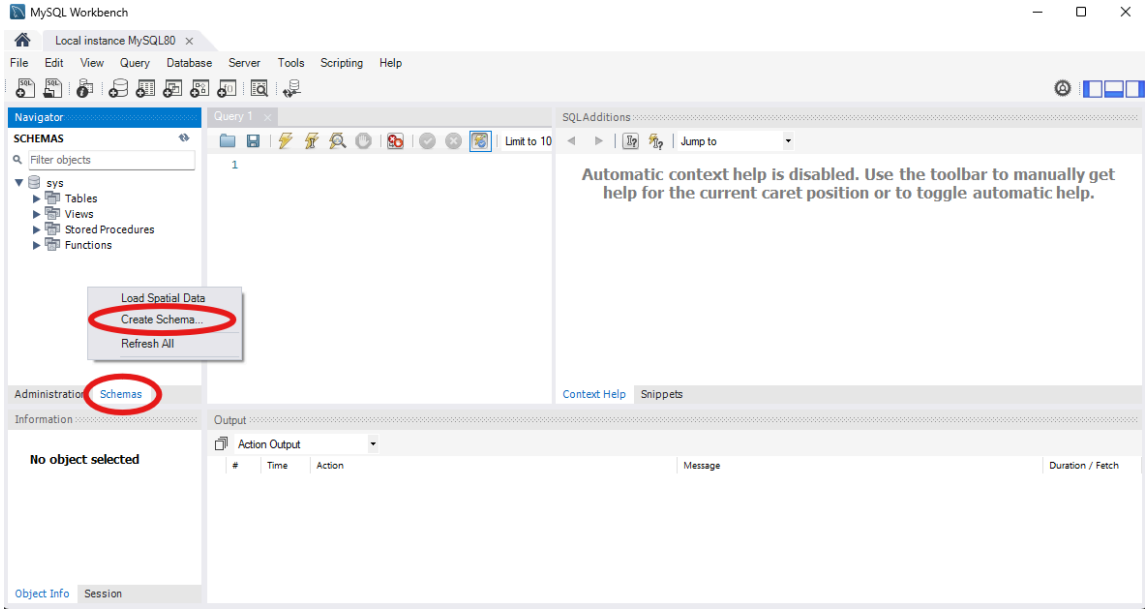
- The Server is what holds your databases, tables, entries.
- The Workbench is a program that you will use to look at the tables and verify that your flask app successfully stored data there.

Creating a database in MySQL Workbench

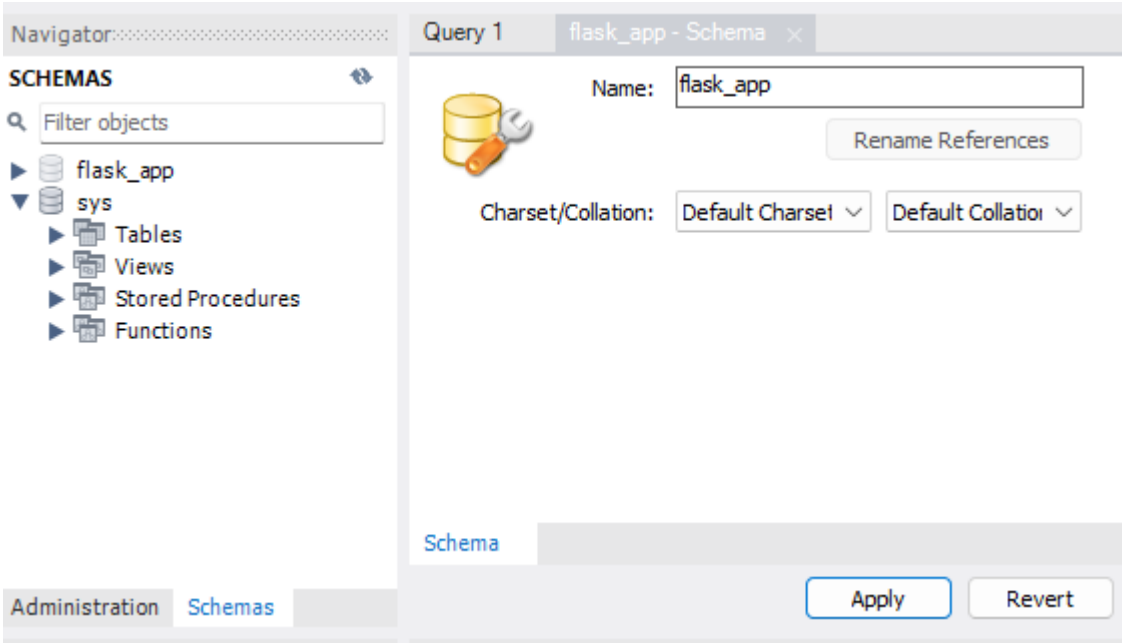
You will create a new database for each project. To create a new database, open the MySQL Workbench program and connect to your server (local instance) with the password you set during the MySQL installation.



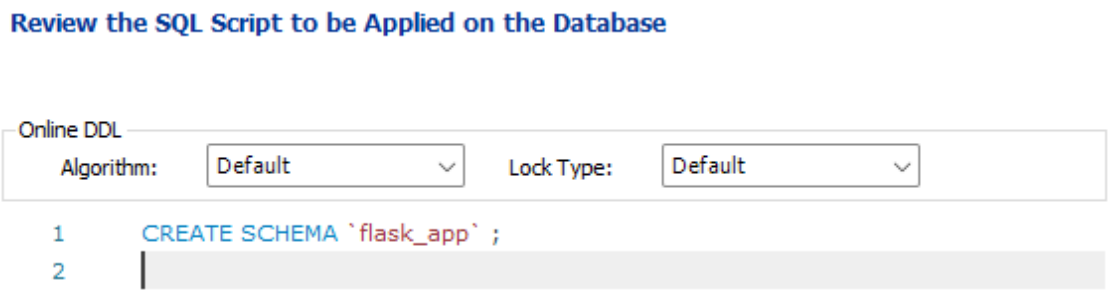
Then, on the left, switch to "Schemas", right click, and create a new schema.



Name the database. Because you will create a new database for each project, give it a more unique name than "flask_app".



MySQL Workbench will create the simple SQL script that creates the new database (schema). Execute that script and see your new database show up in the left panel.



You will not add or manipulate tables through Workbench. The next sections will show you how to do that in Flask.

Configuring Flask-SQLAlchemy

Flask-SQLAlchemy is an object-relational mapper (ORM) that lets you interact with databases using Python objects instead of raw SQL queries. Install the required packages:

```
pip install flask flask-sqlalchemy pymysql
```

You will later connect your Flask app to the database with a code snippet similar to this:

```
app.config['SQLALCHEMY_TRACK_MODIFICATIONS'] = False
app.config['SQLALCHEMY_DATABASE_URI'] =
'mysql+pymysql://root:mysqlrootpassword@localhost:3306/flask_app'
app.config['SECRET_KEY'] = 'your-secret-key-here'
```

In the above code, `app` is the Flask app instance. `.config['X'] = Y` sets a config variable X to a value Y. The imported SQLAlchemy library accesses this config to retrieve the database URI.

You will need to update the exact value for your own project. The *database connection string* is constructed as follows:

```
mysql+pymysql://username:password@hostname:port/database_name
```

- `mysql` is the database "dialect"
- `pymysql` is the database driver
- `root` is the username
- `password` is your root password
- `hostname` and `port` are your URL (localhost during development)
- `database_name` will be whatever you set as the name when you created the database

The secret key is used to cryptographically sign all communications. You can set it to any value.

Creating database models

You do not create tables in the MySQL Workbench. Instead, you will create tables and columns through the Flask app. To create a table, you create a data model as a Python class. Save the following code as your `app.py` and make sure you understand every single line of it:

How does the Python object relate to the database model?

- Each Python class inheriting from `db.Model` creates a table in your database.
- Each instance of that class creates a row.

app.py

```python
from flask import Flask, render_template
from flask_sqlalchemy import SQLAlchemy

app = Flask(__name__)

app.config['SQLALCHEMY_TRACK_MODIFICATIONS'] = False
app.config['SQLALCHEMY_DATABASE_URI'] =
'mysql+pymysql://root:mysqlrootpassword@localhost:3306/flask_app'
app.config['SECRET_KEY'] = 'your-secret-key-here'

db = SQLAlchemy(app)

class User(db.Model): 1
    id = db.Column(db.Integer, primary_key=True) 2
    username = db.Column(db.String(80), unique=True, nullable=False) 3
    email = db.Column(db.String(120), unique=True, nullable=False)

with app.app_context():
    db.create_all() 4
```

1. The user model inherits from the `db.Model` superclass, telling the database to create a table for it.
2. This creates a column with the datatype integer and sets it as the primary key.
3. This creates a column with the datatype string that has to be unique, cannot be null (i.e., left out), and has a max length of 80 characters.
4. This creates all the tables when the app starts.

Choose your column types carefully!

Use integers for IDs because they can easily be incremented and searched. Set them as primary key so that you can reference users in other tables. Think about which other columns have to be unique and non-null. Consider limiting the length of strings. Read more about the available configurations below

SQLAlchemy column options

SQLAlchemy supports these common datatypes for columns:

Column Type	Description	Example
db.Integer	Whole numbers	id = db.Column(db.Integer)
db.String(n)	Text with max length n	name = db.Column(db.String(80))
db.Text	Long text (no length limit)	bio = db.Column(db.Text)
db.Float	Decimal numbers	price = db.Column(db.Float)

Column Type	Description	Example
db.Boolean	True/False values	active = db.Column(db.Boolean)
db.DateTime	Date and time	created = db.Column(db.DateTime)

You might need these common parameters for your project:

- `primary_key=True`: Makes this column the primary key
- `unique=True`: Values must be unique across all rows
- `nullable=False`: Column cannot be empty (required field)
- `default=value`: Sets a default value if none provided

Flask Shell Operations

You can interact with your database directly from the Flask shell. This is useful for testing and debugging. In this command-line interface you write python code line by line into your terminal.

1. Start the Flask shell with `flask shell` instead of `flask run` to start your app. This allows you to continue working in the terminal.
2. Create a new user

   ```
   new_user = User(username='john', email='john@example.com') ❶
   db.session.add(new_user) ❷
   db.session.commit() ❸
   ```

   ❶ This creates a new User object.

   ❷ This adds this user to the database does not yet commit (save) the change.

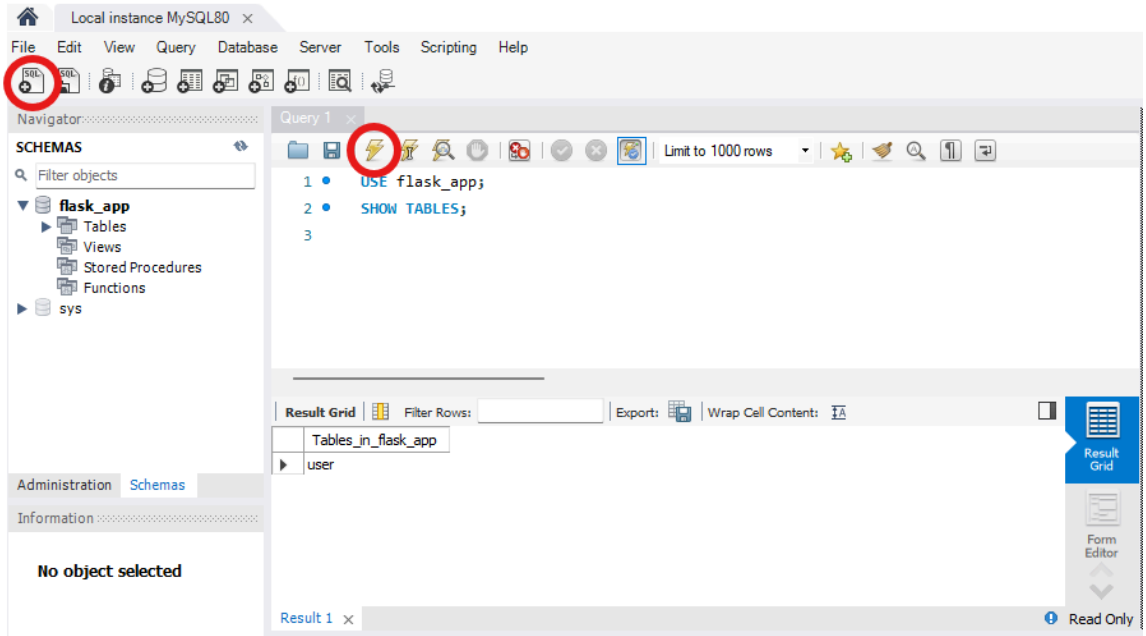
   ❸ This saves the change to the database.
3. Query all users

   ```
   users = User.query.all()
   for user in users:
       print(user.username, user.email)
   ```

   Instead of `.all()` users you could query with filters (e.g., `.filter_by(username='john').first()`)
4. Exit the shell with `exit()`

Verifying changes in MySQL Workbench

You can inspect the database in MySQL Workbench. To create queries, click on "New Query Tab" and type in your query. To execute it, click the flash icon. You will see the results at the bottom. Start by selecting the correct database with `USE flask_app;`



- To view all tables in your database write `SHOW TABLES;`
- To see the structure of a table called `user` write `DESCRIBE user;`
- To see all entries in a table called `user` write `SELECT * FROM user;`
- To count how many entries are in the table `user` write `SELECT COUNT(*) FROM user;`

You can always come back to this program whenever you are unsure about the state of a specific table or the success of an operation.